Annexe A22 — Le chantier B : vectoriser l'étage faces

LibraryV3 · Phase G, étape 4b · de B0 à B4 · 11 au 16 octobre 2026

Cette annexe historise le chantier B jusqu'au pas B4, dans l'ordre où il a été mené. Comme A14, A14a et A15, elle se lit comme une leçon : ce qu'on a supposé, ce qu'on a mesuré, ce qui a été réfuté, et ce qu'il faut retenir. Elle complète A21, qui préparait le chantier, et l'artefact interactif « Lot B4 pas à pas », qui montre le contenu réel des registres étape par étape. Elle répond surtout à une question : **comment traiter 8 triangles à la fois sans changer un seul pixel de l'image ?**

> **Résultat en une ligne.** Sur la scène V1 (32 `sphere_hi`, 508 000 faces), Render passe de 5,80 ms (après B1) à 4,53 ms avec B2, puis de 4,39 à 1,98 ms avec B4 (−55 %, ×2,22, lot du 16 octobre). Les compteurs sont identiques au triangle près sur les trois scènes : la vectorisation est **bit-exacte**. Le levier n'est pas « des registres plus larges » : c'est la suppression des branches et le rangement des données pour que la même instruction traite 8 faces.

1. Le point de départ : pourquoi des scènes V

La Référence L (système solaire, 4 vues) ne soumet plus que 1 388 faces par frame : le LOD et le rendu en point (A19, A20) ont fait leur travail. Un gain sur le traitement des faces y serait invisible. On a donc construit trois scènes de mesure avec `gen_bench_render.py` :

Scène	Contenu	Rôle
`bench_V_V1.json`	32 `sphere_hi`, 508 000 faces soumises par frame	dominée par les faces : **c'est elle qui juge**
`bench_V_V2.json`	une sphère proche, 15 968 faces, beaucoup de pixels	garde-fou : aucune régression quand les pixels dominent
L	`solar_system_v1compat_belt_lod_spheres.json`	témoin : la scène réelle ne doit pas bouger

B0 / B0b. Avant d'optimiser, on découpe. Une sous-zone `RenderVerts` a été placée autour de `ProjectPositions` / `TransformPositions`, après `ClearDepth`. Sur V1, l'étage sommets pesait 11 % de Render et l'étage faces 89 %. La cible était donc l'étage faces.

> **À retenir.** On choisit la scène de mesure pour qu'elle soit dominée par ce qu'on veut améliorer. Mesurer une vectorisation des faces sur une scène qui n'a plus de faces, c'est mesurer le bruit.

2. B1 — Ce que le compilateur fait seul

Premier pas, sans toucher au code : `/arch:AVX2` et `/fp:precise` écrits une seule fois, dans `build/LV3.Common.props`, pour la LIB et l'EXE (règle M6 : un seul foyer). La cible d'erreur A10 se déclenche si le props est introuvable, et aucun `.vcxproj` ne le surcharge.

- **V1 : −26 %** (étage faces −29 %), V2 : −4 %, L : dans le bruit. `RenderVerts` est inchangé et les compteurs sont identiques.
- Une partie du gain vient de `std::floor` / `std::ceil`. En SSE2, il n'existe aucune instruction d'arrondi : le compilateur appelait une fonction. Sous AVX2, ils deviennent un seul `vroundss` (immédiats 1 et 2).

- `/fp:precise` est écrit **explicitement** : il interdit au compilateur de fusionner `a*b + c` en FMA de lui-même (§ 7).

Depuis B1, tout le moteur est encodé en VEX : le compilateur place seul les `vzeroupper`, et les contraintes manuelles du chantier Clear (A15 § 6.3) ont disparu.

3. B2 — Un appel caché dans `std::min({…})`

`TightPixelBox` calculait sa boîte avec `std::min({ a.x, b.x, c.x })`. Dans l'assembleur Release, chaque appel devenait un tableau en pile suivi d'un `call __std_max_element_f_`, et cela quatre fois par appel de `TightPixelBox`. La forme binaire `std::min(a.x, std::min(b.x, c.x))` compile en `vcmpltss` + `vblendvps`, sans appel. MSVC n'émet pas `vminss` sous `/fp:precise`, parce que `minss` et `std::min` ne traitent pas NaN de la même façon.

- Lot appairé V1 (B B2 B2 B) : Render 5,80 → **4,53 ms (−22 %)**, étage faces 4,93 → 3,67 ms (−25,6 %). Compteurs identiques.
- L : −4,3 %, soit environ 1,4 ns par appel supprimé. Sur V1, environ 0,6 ns par appel seulement : le recouvrement hors ordre absorbait déjà une partie du coût.

À retenir. Une extrapolation est une hypothèse. La prédiction faite avec le thermomètre (−1,2 ms) a été confirmée ; l'extrapolation depuis L (−2,9 ms) a été réfutée. Le coût d'une instruction dépend de ce qui l'entoure.

× **Lire le Debug pour juger le Release.** En Debug, `std::min` est toujours un `call` ; cela ne prouve rien. On lit le Release, avec `/FAs` sur le seul `.cpp` concerné.

4. B1b — Où passe le temps : l'ablation

Avant de vectoriser, il fallait savoir ce que coûtaient les **tests** (face arrière, boîte serrée) et ce que coûtait le **dessin** des triangles retenus. Le commutateur `LV3_ABLATION_RASTER` débranche `DrawTriangle` : la différence avec le binaire normal donne le coût du dessin.

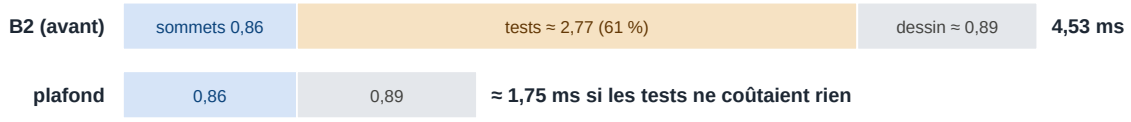


Figure 1 — Découpage de Render V1 sur le binaire B2 (ablation B1b). 508 000 faces testées en ≈ 2,77 ms (5,5 ns par face) ; 18 120 triangles dessinés en ≈ 0,89 ms (49 ns par triangle).

Loi d'Amdahl. Si on divise le coût des tests par 4, Render tombe à ≈ 2,45 ms (×1,85). Même avec des tests gratuits, Render ne descendrait pas sous ≈ 1,75 ms (×2,6) : c'est le plafond du pas B4. Le calcul dit aussi ce qu'il ne faut pas faire tout de suite : l'étage sommets (19 %) ne mérite pas d'effort avant les tests (61 %).

5. B4 — La conception

5.1 Inventaire des 5,5 ns par face

À environ 4,4 GHz, 5,5 ns représentent environ 24 cycles par face. Ce découpage vient de la lecture du code scalaire ; le listing Release l'a ensuite confirmé dans ses grandes lignes.

Poste	Face arrière	Face avant
Adresse `f*vpf`, 3 indices, 6 lectures x/y	≈ 11 uops	≈ 11
`FaceColor(f)`, calculée **avant** le test, même pour les faces rejetées	≈ 6	≈ 6
Aire (4 sub, 2 mul, 1 sub) + comparaison + **branche**	≈ 9	≈ 9
4 chaînes min/max, 4 sub, 4 `vroundss`, 4 `cvttss2si`	—	≈ 40
`ClampBox` + `Empty()` + **branche**	—	≈ 11

Le point décisif tient aux **branches**. Sur V1, environ 93 % des faces avant ont une boîte vide : ce sont des triangles plus petits qu'un pixel, qui tiennent entre deux centres. Les 7 % restantes sont dispersées au hasard, et le prédicteur se trompe autour de chacune. Chaque erreur coûte 15 à 20 cycles.

5.2 Le principe : SIMD vertical, masques, compaction

Une instruction AVX2 applique la même opération aux 8 couloirs d'un registre de 256 bits. On l'utilise **verticalement** : un registre contient la même grandeur pour 8 faces différentes (les 8 x du coin a, par exemple). Les tests produisent alors, au lieu d'un `if` par face, des **masques** : un couloir vaut 0xFFFFFFFF (vrai) ou 0 (faux). Les masques se combinent par un ET bit à bit, puis `movemask` les réduit à 8 bits dans un entier. Le dessin reste scalaire, mais on ne le parcourt que pour les bits à 1.

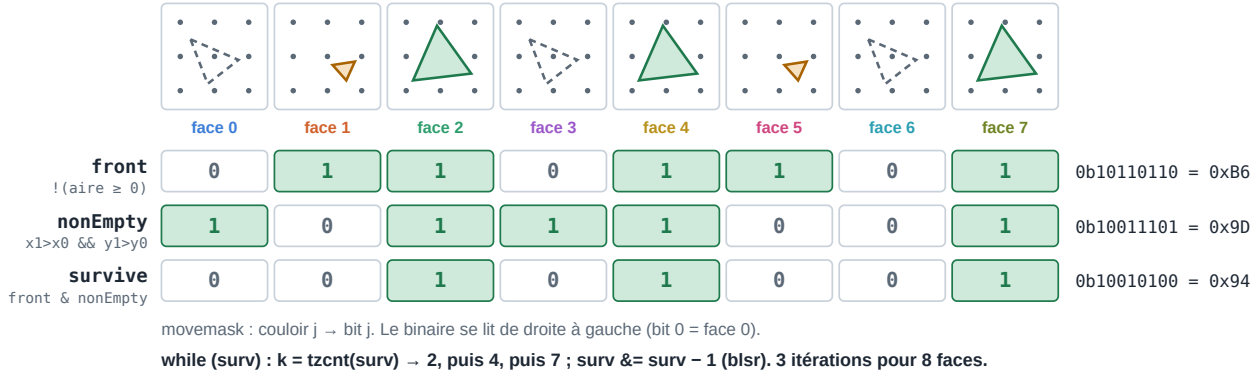


Figure 2 — Un lot de 8 faces. Pointillés : face arrière. Orange : face avant dont la boîte ne contient aucun centre de pixel. Vert : survivante. Les deux tests ne contiennent aucune branche ; la boucle ne visite que les survivantes.

Le pipeline d'un lot se lit comme quatre questions :

Phase	Question	Étapes
1. Rassembler	mettre la face j dans le couloir j	charger, `unpacklo`, `shuffle`
2. Orientation	la face regarde-t-elle la caméra ?	arêtes, aire, masque `front`
3. Boîte de pixels	touche-t-elle un centre de pixel ?	min/max, arrondis, entiers, clamp, masque `nonEmpty`
4. Décider	qui dessiner, et dans quel ordre ?	`movemask`, compaction `tzcnt` / `blsr`

5.3 Rassembler les données : la transposition

Les sommets projetés sont en AoS : `RasterVertex { x, y, z, invW }`, 16 octets alignés. Une face j désigne ses coins par des indices quelconques : `idx[j*3]`, `idx[j*3+1]`, `idx[j*3+2]`. Les 8 `ax` d'un lot sont donc éparpillés en mémoire, et il faut les ranger. Trois étapes y suffisent pour un coin :

1. **Charger.** Une lecture de 16 octets ramène un sommet entier, et un registre de 256 bits en contient deux. Les faces sont appariées (0,4), (1,5), (2,6), (3,7). La raison est que `unpack` et `shuffle` ne font **jamais passer une valeur d'une moitié de 128 bits à l'autre** : la moitié basse doit déjà contenir les faces des couloirs 0 à 3, la haute celles des couloirs 4 à 7. Avec les paires (0,1), la face 1 serait en haut et ne pourrait jamais rejoindre le couloir 1.
2. **`unpacklo`.** Il faut réunir 4 x venus de 4 registres, et aucune instruction ne le fait en une fois. `unpacklo(A, B)` alterne les deux premiers éléments de chaque registre (A0 B0 A1 B1) : les x de deux faces se retrouvent côte à côte, puis leurs y. z et invW sont abandonnés sans instruction supplémentaire.
3. **`shuffle`.** Il prend deux éléments du premier registre et deux du second. `_MM_SHUFFLE(d,c,b,a)` se lit de droite à gauche : (1,0,1,0) prend les éléments 0 et 1 (les x), (3,2,3,2) les éléments 2 et 3 (les y).

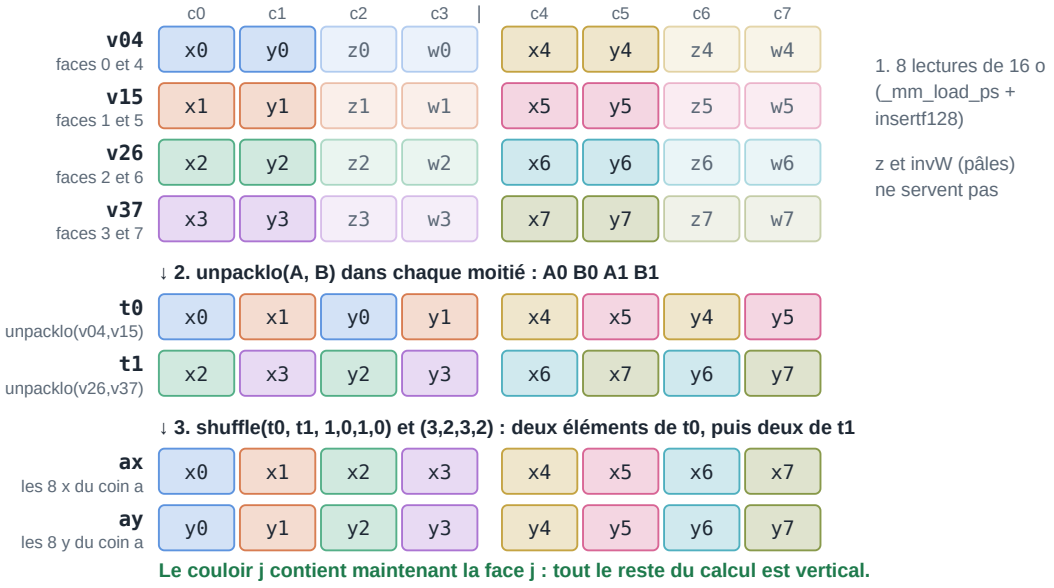


Figure 3 — La transposition du coin a. Chaque couleur est une face. Les cases pâles (z, invW) sont lues avec le sommet mais abandonnées par unpacklo. La même recette est appliquée aux coins b et c.

Pour 8 faces, le rangement coûte 24 lectures de 16 octets, 6 `unpacklo` et 6 `shuffle`. C'est environ les deux tiers des instructions d'un lot : la vectorisation d'une boucle qui calcule peu est d'abord une affaire de données.

5.4 Pourquoi pas le SoA, le gather ou la désindexation

Question naturelle : puisque le rangement coûte cher, pourquoi ne pas ranger les sommets en SoA (`rx[]`, `ry[]`) ? Parce que le SoA n'aide que si l'accès est **séquentiel**. `_mm256_load_ps(rx + i)` charge les x des sommets i à i+7 ; or l'étage faces demande les sommets `idx[0]`, `idx[3]`, `idx[6]`… Ce sont les indices, donc la topologie du mesh, qui éparpillent les données, pas leur rangement.

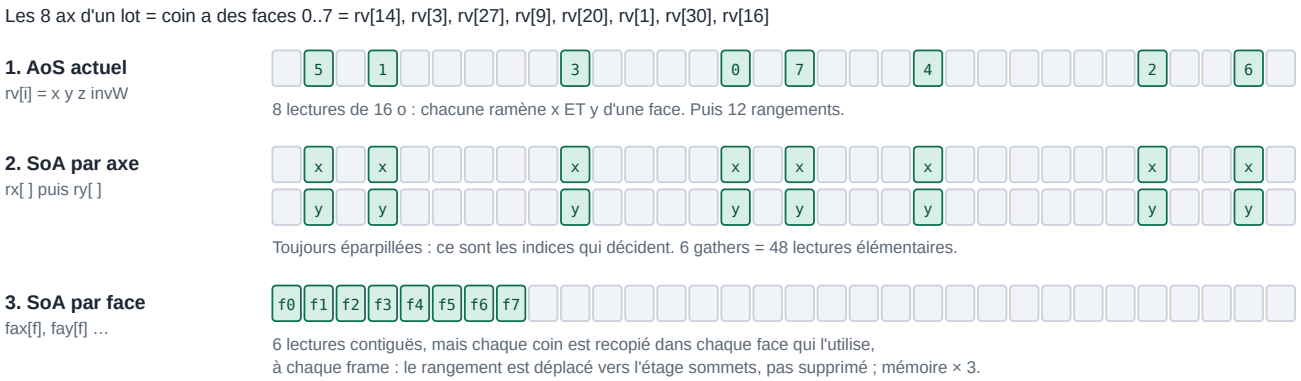


Figure 4 — Où sont les 8 ax d'un lot selon le rangement. Seule la désindexation les rend contigus, au prix d'une recopie par coin à chaque frame.

Rangement	Lectures pour 8 faces	Coût caché
AoS + transposition (retenu)	24 × 16 o	12 rangements ; x et y arrivent ensemble
SoA + 6 gathers	48 × 4 o	désentrelacer 24 indices ; gathers ralentis par le microcode anti-Downfall (GDS) sur l'i9-10900X
SoA par face (désindexé)	6 × 32 o	chaque sommet recopié dans ~6 faces, à chaque frame ; 95 Ko au lieu de 34 Ko pour `sphere_hi`, donc hors du L1 ; contraire à la règle du chantier 2 (« une fois par sommet, jamais par coin »)

À retenir. La règle DOD n'est pas « SoA partout ». On choisit la disposition pour l'algorithme qui la **lit**. Les positions objet, lues séquentiellement par l'étage sommets, passeront en SoA (B3). Les sommets projetés, lus par indices, restent en AoS. Le rangement de l'étage faces est le prix du partage des sommets, qui rapporte bien davantage : un sommet transformé une fois au lieu de six.

5.5 Les deux tests, intrinsèque par intrinsèque

Chaque intrinsèque reproduit une opération du scalaire, dans le même ordre.

Étape	Scalaire	AVX2	Pourquoi cette forme
Arêtes	`ux = ax − cx`, …	4 `_mm256_sub_ps`	forme antisymétrique de `EdgeFunction` : c est soustrait d'abord
Aire	`ux*vy − uy*vx`	`sub(mul, mul)`	deux arrondis puis un, sans FMA
Front	`!(area >= 0)`	`cmp_ps(area, 0, _CMP_NGE_UQ)`	NGE non ordonné : NaN compte comme face avant
Boîte	`std::min(a, std::min(b, c))`	`min_ps(min_ps(c, b), a)`	`std::min(l, r) == min_ps(r, l)` : ordre (droite, gauche)
Centres	`ceil(min − 0,5)`, `floor(max − 0,5)`	`round_ps` (immédiats 10 et 9)	premier et dernier centre de pixel dans la boîte
Entiers	`int32_t(…)`, `+1`	`cvttps_epi32`, `add_epi32`	hors plage ou NaN : INT_MIN des deux côtés ; borne x1 exclusive
Clamp	`ClampBox` (4 `if`)	`max_epi32` / `min_epi32`	en entiers, exact, sans question d'ordre
Non vide	`!(x0 >= x1 || y0 >= y1)`	`and(cmpgt(x1,x0), cmpgt(y1,y0))`	même format de masque que front
Décision	2 branches par face	`movemask_ps` ×2, `and`	8 couloirs → 8 bits ; un lot vide = un entier nul

Un exemple chiffré pour l'étape « centres ». Une face a minX = 3,3 et maxX = 3,4. On calcule 3,3 − 0,5 = 2,8, dont le ceil donne 3 (le premier centre à droite de minX est 3,5) ; puis 3,4 − 0,5 = 2,9, dont le floor donne 2. Le premier pixel (3) est après le dernier (2) : le triangle tient entre deux centres, la boîte est vide. C'est le cas de 93 % des faces avant de V1.

5.6 Le dernier lot, la compaction, les compteurs

- **Pas de queue scalaire.** Quand `nFaces` n'est pas multiple de 8, le dernier lot est complété en répétant la dernière face ; un masque `valid = (1 << n) − 1` efface ensuite les couloirs de bourrage. Il n'y a qu'un chemin de calcul. Le compilateur traduit `& valid` par un seul `bzhi`.
- **Compaction.** `std::countr_zero` donne l'indice du bit à 1 le plus bas (`tzcnt`), et `surv &= surv − 1` l'efface (`blsr`). On visite les survivantes dans l'ordre croissant : `DrawTriangle` est appelé dans le même ordre qu'avant, ce qui compte en cas d'égalité de profondeur.
- **`FaceColor`** n'est plus calculée que pour les survivantes.
- **Compteurs.** `TrisRasterized` = `popcount(front)` et `TrisEarlyRejected` = `popcount(front & ~surv)` gardent exactement leur sens de la Référence F. Ce sont eux qui prouvent la bit-exactitude dans chaque run de mesure.
- **Domaine.** Seuls les triangles (`vpf == 3`) du chemin Inside, hors Wireframe, passent par les lots. Les quads, le Wireframe (qui n'a pas de rejet 3b) et le chemin clippé gardent la boucle scalaire. La soumission commune est extraite dans `SubmitRasterTriangle` : l'ablation M7 n'existe qu'à un endroit.

6. La bit-exactitude, et comment on la prouve

6.1 Pourquoi l'exiger

Le chantier 2b a posé que `ProjectClip` est la seule formule de projection, appelée par le chemin Inside et par le chemin clippé, « donc une image identique au bit près entre les deux chemins ». Si un objet passant de Intersect

à Inside changeait ses pixels de bord, on verrait un clignotement. Un pas de vectorisation n'est donc valide que s'il donne exactement les mêmes décisions que le scalaire, y compris sur NaN, ±0, ±inf et les valeurs hors de la plage int32.

6.2 Trois filets

1. **Banc hors moteur**, avant de livrer le code : 25,6 millions de faces comparées au scalaire. Les sommets sont fabriqués pour être adverses : NaN, ±inf, ±0, demi-entiers exacts (les bords de ceil/floor), valeurs de 1e10 (débordement int32), et 4 viewports dont un nul et un splitté. Résultat : **0 désaccord**.
2. **TNR** `Test_B4_ClassifyFaces8` : 480 000 faces avec une graine fixe, 3 viewports. Le test exige aussi que les trois issues (arrière, vide, survivante) existent, pour ne jamais passer en ne testant rien. Statut : vert.
3. **Filet Debug** dans `EmitTriangleList8` : chaque lot est refait en scalaire et comparé bit à bit, sur les vraies faces de la scène. Run V1 en Debug : aucune assertion.

6.3 Les contre-exemples, mesurés sur le banc

× `_mm256_min_ps(a, _mm256_min_ps(b, c))`, l'ordre « naturel ». Résultat : 378 533 désaccords sur 25,6 M faces. `minps` rend son second opérande quand la comparaison échoue (NaN ou égalité), alors que `std::min` de MSVC rend son premier.

× `_CMP_LT_OQ` au lieu de `_CMP_NGE_UQ`. Résultat : 2 860 792 désaccords. Une aire NaN devient face arrière, alors que le scalaire la traite comme face avant.

Dans les vraies scènes, le chemin Inside garantit w > 0 et ces cas n'apparaissent jamais. C'est précisément pourquoi la TNR doit les fabriquer : une erreur qui ne se manifeste jamais dans les scènes de test se manifestera un jour dans une autre.

7. Pourquoi pas de FMA

Une FMA calcule `a·b + c` avec **un seul** arrondi au lieu de deux. Elle n'est pas fausse : elle est plus précise. Mais elle donne un résultat **différent**, et le rasterizer repose sur des égalités exactes que ce changement casse.

Calcul (a = 0.1f)	Résultat
sans FMA : `a*a − a*a`	**0**, exactement
avec FMA : `fma(a, a, −(a*a))`	**$-4{,}1\cdot10^{-10}$** (l'erreur d'arrondi du produit)

Contre-exemple 1 : le zéro exact disparaît. Pour un triangle dégénéré (deux sommets confondus), l'aire vaut 0 sans FMA et le triangle est rejeté par `IsBackFacing(0)`. Avec FMA, elle vaut $-4{,}1\cdot10^{-10}$: le triangle d'aire nulle passe pour une face avant.

Contre-exemple 2 : l'antisymétrie casse. Deux triangles voisins parcourent leur arête commune en sens inverse. Pour un pixel p exactement sur l'arête, la forme antisymétrique garantit `E(a,b,p) == −E(b,a,p)` au bit près, et la règle top-left attribue p à un seul triangle. Cas trouvé par recherche, avec a = (131.286, 210.714), b = (264, 78), p = (203.5, 138.5) :

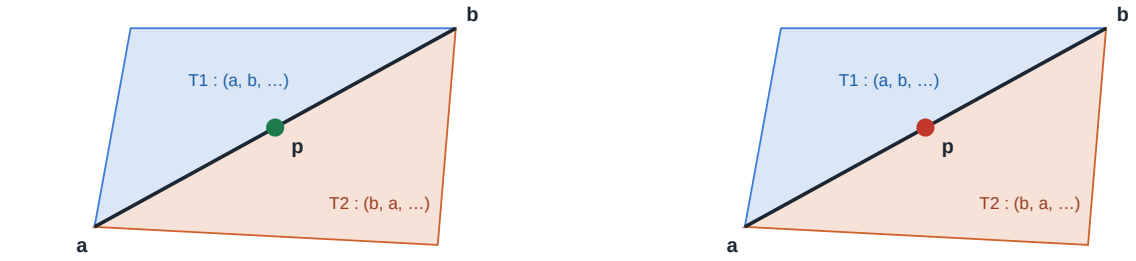


Figure 5 — Une arête partagée. `fma(ux, vy, −(uy·vx))` garde le premier produit exact et arrondit le second ; en inversant a et b, ce n'est plus le même produit qui est arrondi. Le défaut existe même si tout le moteur utilise la FMA.

Contre-exemple 3 : deux chemins qui divergent. Même là où la FMA ne casse aucune propriété, un chemin avec FMA et un chemin sans diffèrent au dernier bit. Un pixel de bord change alors de triangle quand l'objet change de chemin.

Où	FMA ?
`MulRow`, `ProjectClip`, `ToRaster`, éclairage	Oui, plus tard : décision séparée, sur tous les chemins à la fois, nouvelle image de référence acceptée
`EdgeFunction` (face arrière, couverture, barycentriques)	**Jamais** sous la forme `fma(a, b, −c·d)`
Un chemin avec FMA, l'autre sans	**Jamais**

8. Lire l'assembleur Release

Le listing `/FAs` de `RenderSystem.cpp` a été vérifié instruction par instruction avant toute mesure.

Attendu	Trouvé
24 lectures de 16 octets	12 `vmovups xmm` + 12 `vinsertf128 …, [rdi+rcx*8], 1`
Adresse `rv + idx*16`	`add rax, rax` puis `[rdi+rax*8]`
6 unpack, 6 shuffle	`vunpcklps` ×6, `vshufps` immédiats 68 (0x44) et 238 (0xEE)
Aire sans FMA	`vmulps`, `vmulps`, `vsubps`
`_CMP_NGE_UQ`	`vcmpps …, 25` (0x19)
ceil / floor	`vroundps …, 10` et `…, 9`
Entiers, clamp, non vide	`vcvttps2dq`, `vpmaxsd`, `vpminsd`, `vpcmpgtd`, `vpand`
Masques, compteurs, compaction	`vmovmskps`, `bzhi`, `popcnt`, `andn`, `tzcnt`, `blsr`
Appels	seulement `DrawTriangle` pour les survivantes

La preuve de l'ordre min/max se lit dans le listing. `minps` n'étant pas commutatif, le compilateur n'a pas le droit d'inverser ses opérandes. Avec ymm11 = ax, ymm9 = bx, ymm7 = cx, on lit `vminps ymm0, ymm7, ymm9` puis `vminps ymm1, ymm0, ymm11`, c'est-à-dire `min_ps(min_ps(cx, bx), ax)`. Le second produit de l'aire est écrit `vx·uy` au lieu de `uy·vx` : c'est sans conséquence, car la multiplication IEEE est commutative au bit près.

Trois détails, sans défaut :

- Les constantes 0,5 et 1 et les quatre broadcasts du viewport sont rechargés à chaque lot. La cause est le `call DrawTriangle` dans la boucle : sous l'ABI Windows x64, les moitiés hautes des ymm sont volatiles à travers un appel, et le `vzeroupper` les efface. Le coût est d'environ 1,5 instruction par face (piste B4c).
- `_mm_load_ps` devient `vmovups` : sur une adresse alignée, la vitesse est la même. L'assertion d'alignement garde sa valeur de contrat.
- Le prologue sauvegarde xmm6 à xmm12 (registres non volatils de l'ABI), une fois par mesh et par vue.

Budget du chemin chaud : environ 84 instructions de rangement et 50 de calcul, soit **environ 135 instructions pour 8 faces, environ 17 par face**, contre environ 50 par face et deux branches imprévisibles auparavant.

9. La mesure

Protocole. Deux binaires Release : A = B2 (commit `9111851`) et B = B4. i7-1265U épinglé (`affinity=4`), secteur, 1536 × 900, 4 vues, `--bench=300` (60 frames de chauffe, 240 mesurées). Pour chaque scène : un run jetable, puis A1 B1 B2 A2.

9.1 Les compteurs : identiques

Scène	TrisRasterized	TrisEarlyRejected	Survivantes	FacesSubmitted	PointSplats
V1	249 466	231 346	18 120	508 000	0
V2	5 036	12	5 024	15 968	0
L	544	268	—	1 388	1 280

Toutes les colonnes de comptage sont égales entre A et B sur les trois scènes, y compris `MeshesCulled`, `FacesInside`, `VertsTransformed` et `LodLevel*`.

9.2 La recevabilité

`overhead_ns` reste entre 32 et 40 (±30 %). Sur L, `Animation` (39,7 à 40,4 µs) et `LocalXform1` (19,5 à 20,1 µs) sont stables. Sur V1 et V2, ces zones valent environ 1 µs pour une résolution de 100 ns : le critère M19 n'y a pas de sens, et on se fie au thermomètre `RenderVerts` (818, 846, 827 et 827 µs sur V1, dispersion inférieure à 3,5 %).

9.3 Les temps

Scène	Render A (B2)	Render B (B4)	Écart	Render ÷ RenderVerts
V1	4 313 / 4 470 µs	1 969 / 1 982 µs	**−55,0 %**	5,27 → 2,39
V2	1 800 / 1 845 µs	1 715 / 1 713 µs	−5,9 %	63,1 → 59,1
L	191 / 188 µs	183 / 186 µs	−2,6 %	31,9 → 30,6

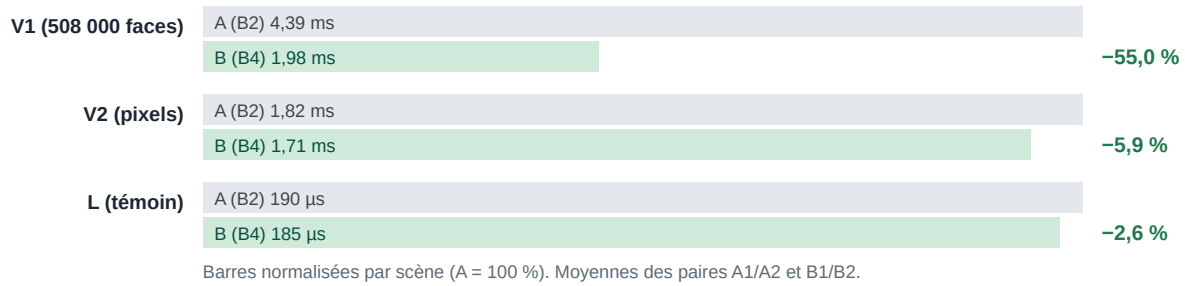


Figure 6 — Render par scène, A (B2) contre B (B4).



Figure 7 — Render V1 avant et après B4. Sommets : mesurés (RenderVerts). Séparation tests / dessin : estimée à partir de l'ablation B1b, le code du dessin étant inchangé.

Sur V1, l'étage faces (Render − RenderVerts) passe de 3,56 à 1,15 ms (−68 %), et la frame de 5,5–5,8 ms à 3,3 ms.

9.4 Une prédiction réfutée par le bas

La prédiction annonçait Render entre 2,3 et 2,7 ms. La mesure donne **1,98 ms**. Le modèle comptait des instructions et sous-estimait deux effets que le listing ne montre pas : le **recouvrement hors ordre** (les lectures d'un lot chevauchent le calcul du précédent) et la **disparition des erreurs de prédiction de branche**. Rapporté au plafond d'Amdahl (≈ 1,70 ms en unités du jour), B4 capture environ 90 % du gain possible. Les tests restants pèsent environ 0,3 ms ; c'est une estimation, sans ablation refaite sur le binaire B4.

À retenir. Une prédiction réfutée dans le bon sens reste une prédiction réfutée. Elle dit que le modèle de coût est incomplet : ici, les branches pesaient plus que les instructions. Le gain d'une vectorisation de tests vient autant des branches supprimées que des 8 couloirs.

10. Les contre-exemples, rassemblés

× Mesurer une vectorisation des faces sur une scène qui n'a plus de faces (§ 1).

× Juger le Release sur l'assembleur Debug (§ 3).

× Extrapoler un coût par appel d'une scène à l'autre (§ 3).

× Charger les sommets en paires (0,1) : unpack et shuffle ne traversent pas les moitiés de 128 bits (§ 5.3).

× Passer en SoA ou au gather pour « éviter » le rangement d'un accès indexé, ou désindexer les faces (§ 5.4).

× Une queue scalaire pour les dernières faces : deux chemins, deux résultats possibles (§ 5.6).

× `min_ps(gauche, droite)` : 378 533 désaccords (§ 6.3).

× `_CMP_LT_OQ` : 2 860 792 désaccords (§ 6.3).

× Un test qui peut passer sans rien tester : la TNR exige que les trois issues existent (§ 6.2).

× La FMA dans `EdgeFunction`, ou dans un seul des deux chemins (§ 7).

× Comparer des temps avant d'avoir comparé les compteurs (§ 9.1).

11. Ouvert : la suite du chantier

Nouveau profil de V1 (Render ≈ 1,98 ms) : étage sommets 0,83 ms (42 %), dessin ≈ 0,86 ms (≈ 43 %), tests ≈ 0,3 ms (≈ 15 %).

Pas	Contenu	Pourquoi
B3	Étage sommets en AVX2, bit-exact. Positions objet en SoA, alignées sur 32 o, bourrées à 8, préparées au chargement ; sortie conservée en AoS `RasterVertex`	prochain pas : premier poste vectorisable de V1
B5	Face arrière en espace objet (caméra ramenée dans le repère de l'objet, plans des faces précalculés)	**conservé** : le moteur doit rester performant sur d'autres scènes, d'autres meshes et d'autres complexités. Mesuré contre B4 ; les pixels de silhouette différents seront comptés
Dessin	Boucle des fragments des survivantes	≈ 43 % de V1, ≈ 94 % de V2
B4c	Constantes et viewport hors de la boucle des lots	≈ 1,5 instruction par face ; à mesurer, sans urgence
B6	AVX-512 par dispatch CPUID, alignement 64 o	i9-10900X seulement
FMA	Décision séparée, sur la transformation et la projection seulement	jamais dans `EdgeFunction`
Option	Run `LV3_RASTER_STATS=1` A/B sur V1	`PixelsCovered` identique ; Σ TrisCov* + TrisEarlyRejected == TrisRasterized

12. Aide-mémoire

Notion	Retenir
Ce qui se vectorise	la même opération sur 8 éléments indépendants ; les données doivent être rangées « verticalement »
Accès indexé	AoS 16 o + transposition (insertf128, unpacklo, shuffle) ; pas de gather
Moitiés de 128 bits	unpack et shuffle ne les traversent pas : paires (0,4) (1,5) (2,6) (3,7)
`_MM_SHUFFLE(d,c,b,a)`	se lit de droite à gauche ; (1,0,1,0) = les x, (3,2,3,2) = les y
Décisions sans branche	masques, ET bit à bit, `movemask` ; compaction `tzcnt` + `blsr`
Bit près	`min_ps(droite, gauche)`, `_CMP_NGE_UQ`, `round_ps` + `cvttps_epi32`, pas de FMA
Dernier lot	répéter la dernière face, masque `valid` (`bzhi`) ; pas de queue scalaire
Preuves	banc adverse, TNR, filet Debug par lot, compteurs identiques, listing Release
Résultat V1	Render 4,39 → 1,98 ms (−55 %) ; étage faces −68 %
Prochain pas	B3 (étage sommets en SoA), puis B5

Annexe — Le code livré

LIB : `Rendering/FaceStage.h` (nouveau), `Scene/RenderSystem.cpp` (`SubmitRasterTriangle`, `EmitTriangleList8`, aiguillage), `LibraryV3.vcxproj` et `.filters`. **EXE :** `Test/Test_B4_ClassifyFaces8.cpp`, `Test/RunAllTests.cpp`.

Rendering/FaceStage.h

```
#pragma once
// ============================================================
//  Rendering/FaceStage.h — l'etage FACES par lots de 8 (chantier B4, phase G)
//
//  ROLE : pour 8 triangles consecutifs, repondre SANS BRANCHE a deux questions :
//    1. la face est-elle AVANT ?            == !IsBackFacing(EdgeFunction(a, b, c))
//    2. sa boite serree est-elle non vide ?  == !TightPixelBox(a, b, c, vp).Empty()
//  Les 8 reponses tiennent dans deux octets de bits (FaceMasks8).
//  Le DESSIN des faces retenues reste scalaire (RenderSystem.cpp).
//
//  LECTURE DES COMMENTAIRES DE REGISTRES
//    Un __m256 contient 8 floats (ou 8 int32), ecrits ici dans l'ORDRE MEMOIRE :
//        [ c0 c1 c2 c3 | c4 c5 c6 c7 ]      c = couloir
//          moitie basse  moitie haute       (128 bits chacune)
//    La barre | rappelle que unpack et shuffle travaillent DANS chaque moitie,
//    jamais a travers. ATTENTION : la doc Intel dessine l'inverse (couloir 7 a gauche).
//
//  REGLE DU BIT PRES
//    Chaque intrinseque reproduit UNE operation du scalaire, dans le MEME ordre :
//    pas de FMA, pas de rcp, min/max dans l'ordre (droite, gauche).
// ============================================================
#include <immintrin.h>    // AVX2 : __m256 (8 float), __m256i (8 int32), _mm256_*
#include <cstdint>
#include "VertexStage.h"  // RasterVertex { x, y, z, invW } : 16 octets, AoS
#include "Viewport.h"

namespace LV3
{
    // Resultat d'un lot. Bit k <-> face k du lot (bit 0 = premiere face du lot).
    struct FaceMasks8
    {
        uint32_t front;     // bit k = 1 : face AVANT (aire < 0, ou NaN)
        uint32_t survive;   // bit k = 1 : face AVANT ET boite non vide (survive ⊆ front)
    };

    // ============================================================
    //  ETAPE 1 — RASSEMBLER x et y du coin k (0 = a, 1 = b, 2 = c) des 8 faces
    // ============================================================
    //  En memoire (AoS), un sommet = 16 octets : x y z invW.
    //  On veut deux registres VERTICAUX :  x = [x0 .. x7]   y = [y0 .. y7]
    //  ou xj = abscisse du coin k de la face j.
    //
    //  Recette : 4 registres de 2 sommets -> 2 unpack -> 2 shuffle.
    //  Les faces sont appariees (0,4) (1,5) (2,6) (3,7) : face j en moitie basse,
    //  face j+4 en moitie haute. Comme unpack/shuffle ne traversent pas la frontiere,
    //  chaque moitie fabrique 4 couloirs du resultat, independamment de l'autre.
    LV3_FORCEINLINE void LoadCornerXY8(const RasterVertex* rv, const uint32_t* idx, int k,
                                       __m256& x, __m256& y) noexcept
    {
        // idx[j*3 + k] = indice du coin k de la face j.
        //   _mm_load_ps            : lit 1 sommet ENTIER (16 o alignes) -> registre 128 bits
        //   _mm256_castps128_ps256 : le pose en moitie BASSE d'un 256 bits (0 instruction)
        //   _mm256_insertf128_ps   : pose un 2e sommet en moitie HAUTE (le dernier argument 1)
        const __m256 v04 = _mm256_insertf128_ps(
            _mm256_castps128_ps256(_mm_load_ps(&rv[idx[0 * 3 + k]].x)),    // face 0 -> bas
                                   _mm_load_ps(&rv[idx[4 * 3 + k]].x), 1); // face 4 -> haut
        const __m256 v15 = _mm256_insertf128_ps(
            _mm256_castps128_ps256(_mm_load_ps(&rv[idx[1 * 3 + k]].x)),    // face 1 -> bas
                                   _mm_load_ps(&rv[idx[5 * 3 + k]].x), 1); // face 5 -> haut
```

```
        const __m256 v26 = _mm256_insertf128_ps(
            _mm256_castps128_ps256(_mm_load_ps(&rv[idx[2 * 3 + k]].x)),    // face 2 -> bas
                                   _mm_load_ps(&rv[idx[6 * 3 + k]].x), 1); // face 6 -> haut
        const __m256 v37 = _mm256_insertf128_ps(
            _mm256_castps128_ps256(_mm_load_ps(&rv[idx[3 * 3 + k]].x)),    // face 3 -> bas
                                   _mm_load_ps(&rv[idx[7 * 3 + k]].x), 1); // face 7 -> haut
        // v04 = [x0 y0 z0 w0 | x4 y4 z4 w4]      (w = invW)
        // v15 = [x1 y1 z1 w1 | x5 y5 z5 w5]
        // v26 = [x2 y2 z2 w2 | x6 y6 z6 w6]
        // v37 = [x3 y3 z3 w3 | x7 y7 z7 w7]

        // unpacklo(A, B), dans CHAQUE moitie : [ A0 B0 A1 B1 ]
        // -> entrelace les x de deux faces, puis leurs y. z et invW sont abandonnes.
        const __m256 t0 = _mm256_unpacklo_ps(v04, v15);   // [x0 x1 y0 y1 | x4 x5 y4 y5]
        const __m256 t1 = _mm256_unpacklo_ps(v26, v37);   // [x2 x3 y2 y3 | x6 x7 y6 y7]

        // shuffle(A, B, imm), dans CHAQUE moitie : [ A[i0] A[i1] B[i2] B[i3] ]
        // _MM_SHUFFLE(d, c, b, a) se lit A L'ENVERS : i0 = a, i1 = b, i2 = c, i3 = d.
        //   (1,0,1,0) -> A[0] A[1] B[0] B[1] : les 2 x de t0, puis les 2 x de t1
        //   (3,2,3,2) -> A[2] A[3] B[2] B[3] : les 2 y de t0, puis les 2 y de t1
        x = _mm256_shuffle_ps(t0, t1, _MM_SHUFFLE(1, 0, 1, 0));   // [x0 x1 x2 x3 | x4 x5 x6 x7]
        y = _mm256_shuffle_ps(t0, t1, _MM_SHUFFLE(3, 2, 3, 2));   // [y0 y1 y2 y3 | y4 y5 y6 y7]

        // A partir d'ici : couloir j == face j. Tout le reste est du SIMD vertical.
    }

    // ═══════════════════════════════════════════════════════════
    //  ClassifyFaces8 — les deux tests pour 8 triangles
    //  idx : 24 indices (3 par face, face j en idx[j*3 .. j*3+2])
    //  rv  : sommets projetes, alignes sur 16 octets (_mm_load_ps)
    // ═══════════════════════════════════════════════════════════
    [[nodiscard]] LV3_FORCEINLINE FaceMasks8 ClassifyFaces8(const RasterVertex* rv,
        const uint32_t* idx, const Viewport& vp) noexcept
    {
        // ── ETAPE 1 : six registres verticaux, couloir j = face j ──────────
        __m256 ax, ay, bx, by, cx, cy;
        LoadCornerXY8(rv, idx, 0, ax, ay);    // coin a des 8 faces
        LoadCornerXY8(rv, idx, 1, bx, by);    // coin b
        LoadCornerXY8(rv, idx, 2, cx, cy);    // coin c

        // ── ETAPE 2 : AIRE SIGNEE — copie de EdgeFunction(a, b, c) ─────────
        // Forme antisymetrique : le point c sert de reference, on le soustrait d'abord.
        const __m256 ux = _mm256_sub_ps(ax, cx), uy = _mm256_sub_ps(ay, cy);
        const __m256 vx = _mm256_sub_ps(bx, cx), vy = _mm256_sub_ps(by, cy);

        // area = ux*vy - uy*vx : deux mul (2 arrondis) puis une sub (1 arrondi),
        // exactement comme le scalaire sous /fp:precise. PAS de _mm256_fmsub_ps.
        const __m256 area = _mm256_sub_ps(_mm256_mul_ps(ux, vy), _mm256_mul_ps(uy, vx));

        // ── ETAPE 3 : MASQUE FRONT ─────────────────────────────────────────
        // Scalaire : IsBackFacing(area) == (area >= 0)  ->  face AVANT == !(area >= 0)
        // _CMP_NGE_UQ = Not Greater-or-Equal, Unordered -> vrai, Quiet (pas d'exception).
        // Donc NaN = AVANT, comme le scalaire. (_CMP_LT_OQ rendrait NaN = ARRIERE : faux.)
        const __m256 front = _mm256_cmp_ps(area, _mm256_setzero_ps(), _CMP_NGE_UQ);

        // ── ETAPE 4 : BOITE FLOTTANTE — copie de TightPixelBox ─────────────
        // MSVC : std::min(l, r) == (r < l) ? r : l
        //        _mm256_min_ps(A, B) == (A < B) ? A : B, et rend B si NaN ou egalite.
        // => std::min(l, r) == _mm256_min_ps(r, l)   : TOUJOURS (droite, gauche).
        const __m256 minX = _mm256_min_ps(_mm256_min_ps(cx, bx), ax);
        const __m256 minY = _mm256_min_ps(_mm256_min_ps(cy, by), ay);
        // MSVC : std::max(l, r) == (l < r) ? r : l == _mm256_max_ps(r, l) : meme regle.
        const __m256 maxX = _mm256_max_ps(_mm256_max_ps(cx, bx), ax);
        const __m256 maxY = _mm256_max_ps(_mm256_max_ps(cy, by), ay);

        // ── ETAPE 5 : CENTRES DE PIXELS COUVERTS, en entiers ───────────────
        //   1er centre  >= minX : ceil (minX - 0.5)  -> x0 (inclusif)
        //   dernier     <= maxX : floor(maxX - 0.5)  -> x1 = floor + 1 (EXCLUSIF)
        const __m256  half = _mm256_set1_ps(0.5f);
        const __m256i one  = _mm256_set1_epi32(1);
        constexpr int kCeil  = _MM_FROUND_TO_POS_INF | _MM_FROUND_NO_EXC;   // == std::ceil
        constexpr int kFloor = _MM_FROUND_TO_NEG_INF | _MM_FROUND_NO_EXC;   // == std::floor
```

```
        // sub (- 0.5)  ->  round (ceil ou floor)  ->  cvtt (troncature en int32)
        // cvttps_epi32 : hors plage ou NaN -> INT_MIN, comme le cvttss2si du scalaire.
        __m256i x0 = _mm256_cvttps_epi32(_mm256_round_ps(_mm256_sub_ps(minX, half), kCeil));
        __m256i y0 = _mm256_cvttps_epi32(_mm256_round_ps(_mm256_sub_ps(minY, half), kCeil));
        __m256i x1 = _mm256_add_epi32(
                        _mm256_cvttps_epi32(_mm256_round_ps(_mm256_sub_ps(maxX, half), kFloor)), one);
        __m256i y1 = _mm256_add_epi32(
                       _mm256_cvttps_epi32(_mm256_round_ps(_mm256_sub_ps(maxY, half), kFloor)), one);

        // ── ETAPE 6 : CLAMP — copie de Viewport::ClampBox, en entiers (exact) ──
        x0 = _mm256_max_epi32(x0, _mm256_set1_epi32(vp.x));
        y0 = _mm256_max_epi32(y0, _mm256_set1_epi32(vp.y));
        x1 = _mm256_min_epi32(x1, _mm256_set1_epi32(vp.x + vp.width));
        y1 = _mm256_min_epi32(y1, _mm256_set1_epi32(vp.y + vp.height));

        // ── ETAPE 7 : MASQUE NON VIDE : (x1 > x0) && (y1 > y0) ──────────────
        const __m256i nonEmpty = _mm256_and_si256(_mm256_cmpgt_epi32(x1, x0),
                                                  _mm256_cmpgt_epi32(y1, y0));

        // ── ETAPE 8 : 8 COULOIRS -> 8 BITS ──────────────────────────────────
        // movemask_ps : bit j = bit de signe du couloir j. castsi256_ps : 0 instruction.
        const uint32_t f = uint32_t(_mm256_movemask_ps(front));
        const uint32_t s = f & uint32_t(_mm256_movemask_ps(_mm256_castsi256_ps(nonEmpty)));
        return { f, s };
    }
}
```

Scene/RenderSystem.cpp — EmitTriangleList8 et aiguillage

```
// Scene/RenderSystem.cpp — CHANTIER B4 : etage FACES par lots de 8
// Domaine : triangles (vpf == 3), chemin Inside, hors Wireframe.
static void EmitTriangleList8(Renderer& renderer, const RasterVertex* rv,
    const uint32_t* indices, size_t nFaces, size_t nVerts, const Viewport& vp,
    bool hasTint, Color tint)
{
    (void)nVerts;   // lu par le seul filet Debug ci-dessous
    LV3_ASSERT((reinterpret_cast<uintptr_t>(rv) & 15u) == 0);   // exige par _mm_load_ps

    uint32_t padIdx[24];   // indices du DERNIER lot quand nFaces n'est pas multiple de 8

    for (size_t base = 0; base < nFaces; base += 8)     // base = 1re face du lot
    {
        // 1. Taille du lot et masque des couloirs VALIDES (n = 8 -> 0xFF, n = 3 -> 0x07)
        const size_t    n     = std::min<size_t>(8, nFaces - base);
        const uint32_t  valid = (1u << n) - 1u;
        const uint32_t* idx   = indices + base * 3;

        // 2. Dernier lot incomplet : PAS de boucle scalaire de queue. On complete
        //    les couloirs n..7 en REPETANT la derniere vraie face ; 'valid' les efface.
        if (n < 8)
        {
            for (size_t j = 0; j < 8; ++j)
            {
                const size_t src = std::min(j, n - 1) * 3;
                padIdx[j * 3 + 0] = idx[src + 0];
                padIdx[j * 3 + 1] = idx[src + 1];
                padIdx[j * 3 + 2] = idx[src + 2];
            }
            idx = padIdx;
        }

        // 3. Les deux tests, 8 faces d'un coup
        const FaceMasks8 m = ClassifyFaces8(rv, idx, vp);

        #if LV3_DEBUG   // filet : CHAQUE lot est refait en scalaire et compare bit a bit
        for (uint32_t k = 0; k < 8; ++k)
        {
            const uint32_t* t = idx + k * 3;
            LV3_ASSERT(t[0] < nVerts && t[1] < nVerts && t[2] < nVerts);
```

```
            const bool fr = !IsBackFacing(EdgeFunction(rv[t[0]], rv[t[1]], rv[t[2]]));
            const bool sv = fr && !TightPixelBox(rv[t[0]], rv[t[1]], rv[t[2]], vp).Empty();
            LV3_ASSERT(fr == bool((m.front >> k) & 1u) && sv == bool((m.survive >> k) & 1u));
        }
        #endif

        // 4. Compteurs : memes definitions que EmitRasterTriangle
        const uint32_t front = m.front   & valid;
        uint32_t       surv  = m.survive & valid;
        LV3_PROF_COUNT(EProfCounter::TrisRasterized,    uint64_t(std::popcount(front)));
        LV3_PROF_COUNT(EProfCounter::TrisEarlyRejected, uint64_t(std::popcount(front & ~surv)));

        // 5. Compaction : seules les survivantes, ordre CROISSANT (= ordre de l'ancienne boucle)
        while (surv)
        {
            const uint32_t k = uint32_t(std::countr_zero(surv)); // tzcnt
            surv &= surv - 1u;                                   // blsr
            const uint32_t* t = idx + k * 3;
            SubmitRasterTriangle(renderer, rv[t[0]], rv[t[1]], rv[t[2]],
                                 hasTint ? tint : FaceColor(int(base + k)));
        }
    }
}

// Aiguillage, dans RenderView, chemin Inside, apres ProjectPositions :
if (vpf == 3 && view.mode != ERenderMode::Wireframe)
{
    EmitTriangleList8(renderer, rv, mesh->indices.data(), nFaces, nVerts,
                      view.viewport, hasTint, tint);
    continue;
}
// sinon (quads, Wireframe) : boucle scalaire inchangee
```